M3C2 Tool

User Documentation

CHECKPOINT 13

Point Cloud Change Detection via M3C2 Algorithm

Built on py4dgeo · laspy · NumPy · SciPy · matplotlib

March 2026

Samuel Lebó

Table of Contents

1. Overview

The M3C2 Tool is a standalone desktop application for detecting and quantifying surface changes between two registered 3-D point clouds. It implements the **Multiscale Model to Model Cloud Comparison (M3C2)** algorithm (Lague et al., 2013) and wraps it in a tkinter GUI with output options suitable for academic and professional geospatial workflows.

1.1 Key Capabilities

- Full M3C2 pipeline: load → subsample → compute distances → significance testing → export
- Automatic parameter estimation from average point spacing (Lague et al. guidelines)
- LAS / LAZ output with extra scalar fields: M3C2_DISTANCE, SIGNIFICANT, Nz, Nxy, DX, DY, DZ, LOD, SPREAD1, SPREAD2
- GeoTIFF raster interpolation of any scalar field (IDW, TIN, Nearest, Kriging)
- DEM export from the corepoint Z values
- Quiver export in two modes: normal-direction and motion vector (PNG, interactive HTML, GPKG)
- Profile cut export: two-panel PNG with raw cloud cross-section (reference blue, comparison red) and M3C2 distance plot
- Interactive profile picker with hillshaded top-down terrain view
- Fully thread-safe GUI – heavy computation never blocks the interface

1.2 Algorithm Reference

Lague, D., Brodu, N., Leroux, J. (2013). Accurate 3D comparison of complex topography with terrestrial laser scanner: Application to the Rangitikei canyon (N-Z). *ISPRS Journal of Photogrammetry and Remote Sensing, 82, 10–26.*

2. Installation & Requirements

2.1 Python Version

Python **3.10 or newer** is required. The tool is tested on Python 3.11 (Windows 10 / 11).

2.2 Required Packages

Package	Minimum Version	Purpose
py4dgeo	≥ 0.6	M3C2 algorithm core
laspy	≥ 2.5	LAS / LAZ read-write
numpy	≥ 1.24	Array computation
scipy	≥ 1.11	KD-tree, interpolation, statistics
matplotlib	≥ 3.7	Profile PNGs, quiver PNGs
plotly	≥ 5.18	Interactive quiver HTML (optional)
geopandas	≥ 0.14	Quiver GPKG export (optional)
shapely	≥ 2.0	Quiver GPKG geometry (optional)
rasterio	≥ 1.3	GeoTIFF raster export (optional)
pykrige	≥ 1.7	Kriging interpolation (optional)

2.3 Installation Command

```
pip install py4dgeo laspy[laszip] numpy scipy matplotlib plotly geopandas shapely
rasterio
```

2.4 Launching the Tool

```
python m3c2_checkpoint13.py
```

> ℹ️ The tool opens a scrollable GUI window. On high-DPI screens you may need to adjust your system DPI scaling if text appears too small.

3. GUI Overview

The main window is organised into collapsible labelled sections. Scroll down with the mouse wheel to see all options. The **Run M3C2 Analysis** button at the bottom starts the pipeline.

Section	Description
Input / Output Files	Paths to reference LAS/LAZ, comparison LAS/LAZ, and output LAS/LAZ
M3C2 Parameters	Cylinder radius, normal radii, corepoint spacing, max depth, confidence level, optional manual sigma
Output Options	TXT export toggle and LAS extra-field selection
Raster Export	Interpolate any scalar field to GeoTIFF
DEM Export	Interpolate corepoint Z values to GeoTIFF
Quiver Export	Normal-direction quiver and/or motion vector arrows
Profile Cut Export	Hillshaded picker + optional TXT polyline → two-panel profile PNG
Progress Bars	Current task and overall pipeline progress

3.1 Progress Indicators

Two progress bars are shown: the **Current Task** bar shows percent completion of the active sub-step, and the **Overall Progress** bar tracks the nine major pipeline steps.

Progress updates are posted back to the main thread asynchronously — updates appear within milliseconds but do not block computation.

4. Input / Output Files

Field	Description
Reference LAS/LAZ	Epoch 1 (older) point cloud – used as the reference surface for normal estimation
Comparison LAS/LAZ	Epoch 2 (newer) point cloud – distances are measured toward this cloud
Output LAS/LAZ (save)	Path and name for the output corepoint cloud; use .laz for compressed output

⚠️ Both input files must be in the same projected coordinate reference system (CRS). No CRS validation is performed by the tool — misregistered clouds will produce meaningless results.

5. M3C2 Parameters

5.1 Parameter Descriptions

Parameter	Unit	Default	Description
Cylinder radius	m	2.0	Radius of the projection cylinder used to collect points for distance measurement. Larger values smooth noise; smaller values preserve local detail.
Normal radii	m	1.5, 2.5, 3.5	Comma-separated list of radii passed to py4dgeo for multi-scale normal estimation. py4dgeo selects the optimal scale automatically.
Corepoint spacing	m	1.0	Minimum distance between corepoints (spatially subsampled from the reference cloud). Set to 0 or tick "Use all points" to skip subsampling.
Max depth detection	m	8.0	M3C2 distances larger than this value are set to NaN (noise filter). Set to 0 to disable.
Confidence level	—	0.95	Significance threshold (0.90 / 0.95 / 0.99). Controls the t-critical multiplier applied to σ_1 and σ_2.
Sigma 1 / Sigma 2	m	auto	Point position uncertainty of each epoch. When "Use manual sigma" is off, these are estimated from py4dgeo SPREAD1/SPREAD2 values.

5.2 Guess Parameters Button

Clicking **Guess Parameters** ▶️ reads the reference LAS/LAZ file (up to 5 000 random points), computes the average nearest-neighbour distance `d_avg`, and fills in sensible starting values:

Field	Formula	Rationale
Cylinder radius	2 × d_avg	Wider than the point spacing to collect enough neighbours
Normal radii	3, 5, 8 × d_avg	Multi-scale range around the spacing (Lague et al. recommendation)
Corepoint spacing	2 × d_avg	Slight decimation keeps computation manageable

> ℹ️ Guess Parameters is a starting point, not a substitute for domain knowledge. For urban / building facades with heterogeneous point density, manual tuning of the cylinder radius is recommended.

5.3 Significance Testing

A change is marked as significant if:

$$|\text{M3C2 distance}| > t_\text{critical} \times \sqrt{\sigma_1^2 + \sigma_2^2}$$

Confidence	t-critical
0.90	1.645
0.95	1.960
0.99	2.576

6. Output Options

6.1 LAS / LAZ Extra Fields

The output corepoint cloud always contains **M3C2_DISTANCE** and **SIGNIFICANT**. The following optional fields can be individually toggled on or off:

Field	Type	Description
M3C2_DISTANCE	float32	Signed distance from epoch 1 to epoch 2 along the surface normal [m]
SIGNIFICANT	int8	1 = change exceeds the detection limit; 0 = not significant
Nz	float32	Absolute Z-component of the unit normal (1 = horizontal surface, 0 = vertical)
Nxy	float32	Horizontal magnitude of the unit normal $\sqrt{(Nx^2+Ny^2)}$
DX	float32	M3C2 displacement projected onto X axis [m]
DY	float32	M3C2 displacement projected onto Y axis [m]
DZ	float32	M3C2 displacement projected onto Z axis [m]
LOD	float32	Level of Detection from py4dgeo [m]
SPREAD1	float32	Point cloud roughness / spread for epoch 1 [m]
SPREAD2	float32	Point cloud roughness / spread for epoch 2 [m]

6.2 TXT Export

When "Also save results to TXT" is ticked, a tab-delimited text file is written alongside the LAS output. The header row lists all field names. This file is directly importable into CloudCompare via **File → Open → ASCII**.

7. Raster and DEM Export

7.1 Raster Export (Scalar Fields → GeoTIFF)

Any scalar field stored on the corepoints can be interpolated to a regular GeoTIFF grid. This is useful for producing M3C2 distance maps suitable for cartographic presentation in QGIS or ArcGIS.

Setting	Description
Resolution [m]	Output cell size in the same units as the CRS. For ALS data at 1 m corepoint spacing, 0.5–1.0 m is typical.
Interpolation	Method used to fill the grid (see §7.3 below).
Fields	Checkboxes for each scalar field. M3C2_DISTANCE is selected by default.

7.2 DEM Export (Z Values → GeoTIFF)

Exports the elevation (Z coordinate) of the corepoints to a GeoTIFF. This is a surface model of the reference epoch derived from the M3C2 corepoint positions — not a raw point cloud DEM.

7.3 Interpolation Methods

Method	Description	Best For
IDW	Inverse Distance Weighting (k=12 neighbours, power=2)	General use; fast; no extrapolation artefacts
TIN	Linear triangulation (scipy griddata)	Preserves break lines; may gap over holes
Nearest	Nearest-neighbour assignment	Categorical fields (SIGNIFICANT); very fast
Kriging	Ordinary kriging with linear variogram (pykrige)	Smooth fields with spatial autocorrelation; slowest

⚠️ Kriging requires pykrige to be installed. It is significantly slower than IDW for large datasets (>50 000 corepoints). Consider using IDW for initial exploration.

8. Quiver Export

Quiver plots visualise the spatial pattern of change as arrow fields. Each arrow represents the mean displacement of corepoints within one grid cell. Two independent modes are available and can both be run in a single analysis.

8.1 Export Modes

Mode	Arrow Direction	Arrow Length	Best For
Normal-direction quiver (_quiver_normal.*)	XY projection of the surface normal, normalised to unit length	|M3C2 distance|	Steep surfaces (walls, embankments) where the horizontal normal component is large
Motion vector (DX · DY) (_quiver_motion.*)	True horizontal displacement vector (DX, DY)	$\sqrt{(DX^2+DY^2)}$	Flat terrain (subsidence, soil creep) where motion has a physical XY component

> ℹ️ On flat surfaces (Nxy ≈ 0), the normal-direction quiver produces near-zero arrows even when there is real vertical change. Use the motion vector mode or inspect the DZ raster for vertical subsidence.

8.2 Grid Cell Size

The grid cell is completely independent of the corepoint spacing. All corepoints whose XY coordinate falls within a cell are averaged. A cell size of 2–5 × the corepoint spacing is usually appropriate. Smaller cells may produce isolated arrows; larger cells smooth spatial variability.

8.3 Output Formats

Format	Filename Suffix	Description
PNG	_quiver_normal.png / _quiver_motion.png	High-resolution static matplotlib figure (1400 px wide, dark background). Suitable for thesis figures.
HTML	_quiver_normal.html / _quiver_motion.html	Interactive Plotly figure. Pan, zoom, hover to read individual cell values. Requires an internet connection to load the Plotly CDN.
GPKG	_quiver_normal.gpkg / _quiver_motion.gpkg	GeoPackage with one LineString per grid cell. Load in QGIS → Layer Properties → Symbology → Arrow. Style by mean_m3c2 or horiz_mag.

8.4 GPKG Attributes

Attribute	Available In	Description
mean_m3c2	Both modes	Mean M3C2 distance for all corepoints in the cell [m]
abs_m3c2	Both modes	Absolute value of mean_m3c2 [m]
cell_x/y	Both modes	Cell centre coordinates [m]
azimuth_deg	Both modes	Arrow azimuth in degrees
n_points	Both modes	Number of corepoints contributing to the cell
dx_mean	Motion only	Mean horizontal DX displacement in the cell [m]
dy_mean	Motion only	Mean horizontal DY displacement in the cell [m]
horiz_mag	Motion only	$\sqrt{(dx_mean^2+dy_mean^2)}$ – unsigned horizontal displacement magnitude [m]

9. Profile Cut Export

Profile cuts project both point clouds and the M3C2 corepoints onto a user-defined polyline and export a two-panel publication-ready PNG per line.

9.1 Profile Output Layout

Panel	Content
Upper (elevation)	Reference cloud points projected into the corridor shown in blue; comparison cloud in red. Point size 3 pt, alpha 0.55–0.65. Legend shows point count per cloud.
Lower (M3C2 distance)	Corepoints coloured by M3C2 distance (RdBu_r diverging scale). Grey = not significant; coloured = significant. Dashed ±LoD threshold lines. Unchanged from previous versions.

9.2 Corridor Width

Both raw clouds and corepoints within **± corridor_width / 2** metres of the polyline are included. A narrow corridor (e.g. 0.5–1 m) gives a clean vertical cross-section; wider corridors show more points but the upper panel can look broad.

> ⚠️ Raw cloud projection can be slow for very dense clouds. Each cloud is capped at 500 000 random points before projection. If your cloud is sparser than this, all points are used.

9.3 Interactive Profile Picker

Click ▶️ **Pick profiles interactively…** to open the hillshaded terrain viewer (reference cloud only). The viewer:

- Bins all reference cloud points to a Z-grid and computes an analytical hillshade (NW light, 45° altitude) identical to QGIS/GDAL
- Blends the hillshade with the terrain colormap for clear edge and ridge visibility
- Allows pan/zoom via the matplotlib toolbar before picking
- Grid resolution (0.25 m → 5 m) is adjustable via a slider – coarser is faster, finer shows more detail
- Picks are shown as orange-red lines with length labels (P1 47.3 m)

Control	Action
Left-click (toolbar off)	Place first point of a line
Left-click again	Place second point – line is saved immediately
↩️ Undo	Remove the last saved line (view is preserved)
✕ Clear all	Remove all saved lines after confirmation
✓ Done	Accept all lines and close the picker
✕ Cancel	Discard all picks and close

Control	Action
Slider (Grid res)	Change hillshade resolution; cached per value so re-sliding is instant

9.4 TXT Import (Fallback)

A profile polyline can also be loaded from a plain text file. Format: one vertex per line, **Y (Northing) X (Easting)** separated by whitespace, comma, or semicolon. Lines beginning with # are ignored. The TXT profile is appended after any interactively picked lines.

```
# Example profile.txt 6234812.4  506670.8 6234587.2  506401.4
```

9.5 Output File Naming

One PNG per profile line is saved alongside the output LAS file:

```
<output_stem>_profile_01.png <output_stem>_profile_02.png ...
```

10. Processing Pipeline

When **Run M3C2 Analysis** is clicked, all GUI values are read on the main thread and passed to a background worker. The nine pipeline steps are:

Step	Name	Description
1	Load point clouds	py4dgeo.read_from_las() reads both LAS/LAZ files into memory. KD-trees are built automatically.
2	Subsampling	Spatially subsample the reference cloud to corepoints using a minimum-distance greedy KD-tree filter. Disabled if spacing = 0 or "Use all points" is ticked.
3	M3C2 distances	py4dgeo.M3C2 estimates normals and measures distances from epoch 1 to epoch 2 at each corepoint.
4	Normal vectors	Independent PCA-based normals are computed using the reference cloud KD-tree (used for DX/DY/DZ and quiver export).
5	Max depth filter	Distances exceeding the threshold are set to NaN.
6	Significance	The Level of Detection (LoD) is computed and each corepoint is flagged SIGNIFICANT (1) or not (0).
7	Normal components	DX, DY, DZ are computed by projecting M3C2 distance onto the unit normal vector.
8	Save LAS/LAZ	Output LAS file is written with all selected scalar fields.
9	Statistics + exports	Console statistics are printed. Rasters, DEM, quiver files, and profile PNGs are generated.

11. Scalar Field Reference

11.1 Normal Vector Components

Normals are estimated independently from M3C2 (via PCA on the reference cloud KD-tree, k=10 neighbours). The sign convention may differ from py4dgeo normals but is consistent across all output fields.

Symbol	Formula	Interpretation
Nz	|nz|	1.0 = perfectly horizontal surface; 0.0 = perfectly vertical surface
Nxy	$\sqrt{(nx^2 + ny^2)}$	0.0 = perfectly horizontal; 1.0 = perfectly vertical. $Nz^2 + Nxy^2 = 1$
DX	dist × nx	East component of displacement [m]
DY	dist × ny	North component of displacement [m]
DZ	dist × nz	Vertical component of displacement [m]. Positive = surface rose.

11.2 py4dgeo Uncertainty Fields

Field	Source	Interpretation
LOD	py4dgeo lodetection	Level of Detection at each corepoint: $t \times \sqrt{(\sigma_1^2 + \sigma_2^2)}$ [m]
SPREAD1	py4dgeo spread1	Roughness / standard deviation of epoch 1 points in the cylinder [m]
SPREAD2	py4dgeo spread2	Roughness / standard deviation of epoch 2 points in the cylinder [m]

12. Recommended QGIS Workflow

12.1 Loading the Corepoint Cloud

1. Drag the output .laz file into QGIS.
2. Set the CRS to match your data (e.g., EPSG:8353 for S-JTSK/Krovak East North).
3. Open Layer Properties → Symbology → Point Cloud.
4. Set Attribute to M3C2_DISTANCE, colormap to RdBu (diverging), centred at 0.
5. Set the range to ±LoD or ±1 SD to highlight significant changes.

12.2 M3C2 Distance Raster

6. Load the _M3C2_DISTANCE.tif file.
7. Properties → Symbology → Singleband pseudocolor, colormap RdBu_r, min/max from actual data.
8. Tick "Clip to extent" and set the min/max manually to ±(detection limit) to saturate noise.

12.3 Quiver Arrows in QGIS

9. Load the _quiver_motion.gpkg file (or _quiver_normal.gpkg).
10. Properties → Symbology → select Rule-based or Single Symbol.
11. Change layer type to Arrow (available in the line renderer).
12. Set fill colour to a graduated scale based on mean_m3c2 using the same diverging colormap.
13. Set arrow width to a fixed value (0.3–0.5 mm) or data-defined from abs_m3c2.

> ✓ For the motion vector GPKG, use horiz_mag for arrow width/size and mean_m3c2 for colour. This way you see both the magnitude of horizontal movement and whether it was positive or negative.

13. Thread Safety

All computation runs in a daemon thread. The main thread runs only the Tcl/Tk event loop. The following rules are enforced to prevent the `Tcl_AsyncDelete` crash that occurs when tkinter objects are accessed from the wrong thread:

Rule	Implementation
GUI values read before thread launch	start_m3c2_threaded() reads all tk.Variable.get() and Entry.get() values on the main thread and passes plain Python values to the worker.
Dialogs posted via root.after()	_ui_error() and _ui_info() use root.after(0, fn) to show messagebox dialogs on the main thread.
Progress bar updates via root.after()	ProgressManager.update_current() and update_overall() schedule all widget writes through root.after(0, fn).
btn_start re-enabled via root.after()	The finally block calls root.after(0, lambda: btn_start.config(state="normal")).
matplotlib backend locked to Agg	matplotlib.use("Agg") is the very first matplotlib call, preventing TkAgg from being set as the global backend by the ProfilePicker import.

ℹ️ If you see RuntimeError: main thread is not in main loop in the console output, check that you are running m3c2_checkpoint13.py and not an older checkpoint file.

14. Troubleshooting

Symptom	Likely Cause	Solution
Tcl_AsyncDelete crash	Running an older checkpoint file	Run m3c2_checkpoint13.py
ImportError: No module named py4dgeo	py4dgeo not installed	pip install py4dgeo
No corepoints found within corridor	Wrong TXT vertex order or mismatched CRS	Check TXT uses Y X (Northing, Easting); verify both clouds share the same CRS
Quiver arrows all point the same direction	Normal-direction mode on flat terrain	Switch to motion vector mode
Raster export fails (rasterio)	rasterio not installed	pip install rasterio
Kriging is extremely slow	Large dataset (>50k pts)	Switch to IDW; or increase corepoint spacing to reduce point count
Hillshade picker appears black	Cloud Z values have very small range	Reduce grid resolution slider to 0.25 m; or check the cloud has Z coordinates (not all zeros)
Profile shows no comparison cloud points	Corridor too narrow for the actual overlap	Increase corridor width in the Profile Cut section
LAS write fails	Output path does not exist	The tool creates missing parent directories automatically; check for permission issues on network drives

15. Version Changelog

Checkpoint	Key Changes
CP 13 (current)	Hillshaded profile picker; dual-cloud profile top panel (ref blue / comp red); matplotlib backend locked to Agg; all widget access marshalled via root.after(); ProgressManager fully thread-safe
CP 12	Normal-direction quiver and motion vector (DX·DY) as independent export options; _quiver_normal.* and _quiver_motion.* suffixes; motion GPKG adds dx_mean, dy_mean, horiz_mag attributes
CP 11	Single quiver export (normal-direction only); profile cut export with TXT polyline input; QGIS-style two-panel profile PNG
CP ≤ 10	Core M3C2 pipeline, raster/DEM export, Guess Parameters, scrollable GUI, progress bars, LAS/TXT output

M3C2 Tool – User Documentation – CHECKPOINT 13 – March 2026 – Samuel Lebó